Developing Arrow + DBPS

Doc version: 1.4 (15/Jan/2026)

1. Background: Arrow + DBPS development

To develop the Arrow + DBPS functionality, we have created a development setup which depends on two separate Docker images:

a) The 'Arrow Dev' Docker image, used for
   i) Building Arrow Python and C++ code
   ii) Building and executing the Arrow tests
   iii) Executing the Python app (e.g. `baseApp.py`)

b) The 'DBPS Dev' Docker image, used for
   i) Building the DBPS server and libraries
   ii) Executing DBPS tests
   iii) Hosting and running the DBPS API Server (`./dbps_api_server`)

Note: both for the 'Arrow Dev' as well 'DBPS Dev' Docker images, the development workflow consists of keeping and editing the source code on your local machine (e.g. laptop). The locally hosted code is then mounted as a directory in their respective Docker image.
This setup facilitates and speeds up development, while removing the ambiguity of having to install all the local tooling and libraries in your local machine.

2. Building the Dev Containers

2.1 Building the 'Arrow Dev' Container

Note: the process of building the Docker image will download and build the Arrow code into the image. This is done just as an assurance that the tooling in the image is complete to enable development and testing. After the image has been built, all the development and testing will take place with code hosted on your local machine, via the directory mounted in `/arrowdev`

This section assumes that the Arrow code has been checked out in the `~/dev/arrow` directory and the DataBatchProtectionService code in `~/dev/DataBatchProtectionService`

1. Create the Dockerfile for 'Arrow Dev' (contents in Appendix 1 below)
   a. `local-box$ cd ~/dev/arrow`
   b. `local-box$` (create `Dockerfile`)
2. Build the Docker image

a. `local-box$ cd ~/dev/arrow`
b. `local-box$ docker build -f Dockerfile -t protegrity-arrow-dev .`

3. Instantiate the Docker image and open up a console
   a. `local-box$ cd ~/dev/arrow`
   b. `local-box$ docker run -it --rm \`
      `-v $(pwd):/arrowdev \`
      `-v $(pwd)/../DataBatchProtectionService:/dbps \`
      `protegrity-arrow-dev`
   c. Step (b) will mount the local directories `~/dev/arrow` and `~/dev/DataBatchProtectionService` inside the Docker instance (in `/arrowdev` and `/dbps`, respectively)

2.2 Building the 'DBPS Dev' Container

Please follow the instructions here

3. Developing in the Docker Containers

3.1 Developing the Arrow code

1. Instantiate and open up a shell within the 'Arrow Dev' Docker image
   a. `local-box$ cd ~/dev/arrow`
   b. `local-box$ docker run -it --rm -v $(pwd):/arrowdev protegrity-arrow-dev`
2. Inside the Docker instance, build the C++ and Python code

```
Shell
# building the C++ code

arrow-docker-img$ cd /arrowdev/cpp && \
                  rm -rf build && \
                  mkdir build && cd build && \
                  cmake -GNinja \
                    -DARROW_WITH_SNAPPY=ON \
                    -DCMAKE_BUILD_TYPE=Debug \
                    -DCMAKE_INSTALL_PREFIX=$(pwd)/install \
                    -DARROW_PARQUET=ON \
                    -DPARQUET_REQUIRE_ENCRYPTION=ON \
                    -DARROW_PYTHON=ON \
                    -DARROW_COMPUTE=ON \
                    -DARROW_TESTING=ON \
                    -DARROW_BUILD_TESTS=ON \
```

```
                  .. && \
                ninja install


# building the Python code

arrow-docker-img$ cd /arrowdev/python
arrow-docker-img$ pip install -r requirements-build.txt && \rm -rf build/
arrow-docker-img$ python3 setup.py build_ext --inplace
arrow-docker-img$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/arrowdev/cpp/build/debug
arrow-docker-img$ export DBPA_LIBRARY_PATH=libdbpsLocalAgent.so
```

3. Run the C++ Tests

```
Shell
arrow-docker-img$ cd /arrowdev
arrow-docker-img$ git submodule update --init
arrow-docker-img$ export
PARQUET_TEST_DATA="${PWD}/cpp/submodules/parquet-testing/data"
arrow-docker-img$ export ARROW_TEST_DATA="${PWD}/testing/data"
arrow-docker-img$ cd /arrowdev/cpp
arrow-docker-img$ ctest -L parquet --test-dir build
```

4. Run the Python Tests

```
Shell
arrow-docker-img$ cd /arrowdev/python
arrow-docker-img$ python3 -m pytest
```

3.2 Developing the DBPS code

This section assumes that the DBPS code has been checked out in the `~/dev/DataBatchProtectionService` directory.

In the DBPS repo (`~/dev/DataBatchProtectionService`), the code from your local box is also mounted inside the Docker instance (in `/app`)

1. Instantiate and open up a shell within the ‘DBPS’ Docker image
    a. `local-box$ cd ~/dev/DataBatchProctectionService`
    b.
```
local-box$ docker run -it --rm \
        -v $(pwd):/app \
        -p 18080:18080 \
        dbps_server /bin/bash
```
2. Build and runs tests of the DBPS code (inside the DBPS Docker instance)

```
Shell
dbps-docker-img$ cd /app
dbps-docker-img$ cmake -B build -S . -G Ninja && \
                 cmake --build build --target all_targets
dbps-docker-img$ ctest --test-dir build
```

4. Testing Arrow + DBPS together

In this section, we describe how to test Arrow + DBPS together, using both Docker images.

As a reminder, the set up is as follows

- In the DBPS image, we will
    - Build the DBPS libraries, both the network-based (aka ‘remote’), as well as the in-memory (aka ‘local’) versions.
    - Run the DBPS Server (to be used by the ‘remote’ version).
- The libraries built in the DBPS image are placed in a common directory, which is mounted in the Arrow Dev image. The Arrow code will be pointed to that location.
- When using the ‘remote’ version, there is ‘network’ communication between the two (Arrow Dev and DBPS) containers.

Based on this:

1. Open up separate shells, one for each the ‘Arrow Dev’ and ‘DBPS’ Docker images (steps 2.1.3 and 3.2.1 above)
2. After building the code in DBPS (step 3.2.2 above), start the DBPS Server

```
Shell
dbps-docker-img$ cd /app
dbps-docker-img$ ./build/dbps_api_server
```

3. Inside the ‘Arrow Dev’ image, edit
   `/arrowdev/python/scripts/test_connection_config_file.json`

```JSON
{
  "server_url": "http://192.168.1.86:18080"
}
```

Note: the IP address here cannot be ‘127.0.0.1’, and also it cannot be the internal address of any of the two Docker containers. Instead, the address in `server_url` **must be** the physical IP address of your local dev box (likely 192.168.x.x or 10.x.x.x) . This is required because the ‘network call’ in the Arrow container will be made to the DBPS container, which is ultimately bound to the local dev box’s IP address.

Please see Appendix 2 (Developing Arrow + DBPS) for more details on the `connection_config` file.

4. Set the necessary environment variables in the ‘Arrow Dev’ image.
   a. LD_LIBRARY_PATH must be pointed to the location where the DBPS shared libraries.
   b. DBPA_LIBRARY_PATH must contain the name of the DBPS library to be used (either `libdbpsRemoteAgent.so` or `libdbpsLocalAgent.so`)

```Shell
arrow-docker-img$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/dbps/build/
arrow-docker-img$ export DBPA_LIBRARY_PATH=libdbpsRemoteAgent.so
```

5. Run the python application

```Shell
arrow-docker-img$ python3 /arrowdev/python/scripts/base_app.py
```

5. `configuration_properties`

The `configuration_properties` struct helps define properties specific to the external encryptor (a.k.a "agent"). In particular, in `configuration_properties` we define: (1) the file name of the shared library (e.g. `*.so`) containing the agent implementation, and (2) additional parameters which may be required by the agent once loaded.

```python
configuration_properties = {
     "EXTERNAL_DBPA_V1": {
         "agent_library_path": "libDBPATestAgent.so",
         "agent_init_timeout_ms": "15000",
         "agent_encrypt_timeout_ms": "35000",
         "agent_decrypt_timeout_ms": "35000",
         "connection_config_file_path" : "/path/to/connection_config.json"
     }
 }
```

Properties are defined per type of external encryptor (e.g. `EXTERNAL_DBPA_V1` in the example above).

Properties known (and used) at the time of this writing are:

- `agent_library_path`: the name of the shared library to load the external encryptor from (`libdbpsRemoteAgent.so`). This is read by the C++ code.
- `agent_init_timeout_ms`: maximum amount of time (millis) allows for the agent's init operation to complete (**).
- `agent_encryption_timeout_ms`: maximum amount of time (millis) allowed for the agent's `encrypt()` operation to complete (**).
- `agent_decryption_timeout_ms`: maximum amount of time (millis) allowed for the agent's `decrypt()` operation to complete(**).
- `connection_config_file_path`: contains the location of the file for configuring the remote agent (i.e. `libdbpsRemoteAgent.so`). This is not required for the 'local' agent. See Appendix 2 (Developing Arrow + DBPS) for details.

(**) The three timeout values are enforced by the Executor (see details here: [Working] Code-level External Encryptor + DLL interfaces/contracts)

6. Environment Variables

In this section, we document env variables useful for development, troubleshooting and executing of external-encryptor based applications

Env Variable	Description	Options
PARQUET_DBPA_LOG_LEVEL	Defines the log level of DBPA (i.e. agent) specific code.	TRACE, DEBUG, INFO, WARN, ERROR, FATAL

Appendix

Appendix 1: Docker File for Arrow Development

```
None
# --- start of Dockerfile (arrow dev)
#
# Step 1: Use the Apache Arrow development image as base for the build stage
FROM docker.io/apache/arrow-dev:arm64v8-ubuntu-22.04-cpp AS arrow-dev-image

# Set environment variable for runtime
ENV LD_LIBRARY_PATH=/usr/local/lib
ENV CMAKE_BUILD_PARALLEL_LEVEL=12
ENV PYTHONPATH=/arrowdev/python

# Set working directory
WORKDIR /arrowdev

# Clone the source.
RUN git clone -b main https://github.com/protegrity/arrow.git .

# some utils and necessary packages
RUN apt-get update && \
    apt-get install -y gdb libboost-date-time-dev libboost-dev thrift-compiler  && \
    apt-get install -y valgrind vim less net-tools && \
    rm -rf /var/lib/apt/lists/*

# Build Arrow C++ libraries
# This is done so that we verify that the image can build the code.
# However the cloned/downloaded sources will not be used during the development process
RUN cd /arrowdev/cpp && \
    rm -rf build && \
    mkdir build && cd build && \
    cmake -GNinja \
      -DARROW_WITH_SNAPPY=ON \
      -DCMAKE_BUILD_TYPE=Debug \
```

```
      -DCMAKE_INSTALL_PREFIX=$(pwd)/install \
      -DARROW_PARQUET=ON \
      -DPARQUET_REQUIRE_ENCRYPTION=ON \
      -DARROW_PYTHON=ON \
      -DARROW_COMPUTE=ON \
      .. && \
    ninja install

# Set environment variables for Arrow and Parquet functionality
ENV CMAKE_PREFIX_PATH=/arrowdev/cpp/build/install
ENV ARROW_HOME=/arrowdev/cpp/build/install
ENV LD_LIBRARY_PATH=$ARROW_HOME/lib:$LD_LIBRARY_PATH

# Build Python bindings
WORKDIR /arrowdev/python
RUN pip install -r requirements-build.txt && \
    rm -rf build/ && \
    python3 setup.py build_ext --inplace

#TODO: this should probably be part of requirements.txt
RUN pip install pandas pytest hypothesis

# Test basic Parquet functionality
RUN python3 -c "import pyarrow.parquet as pq; print('Parquet OK')"

# Test Parquet encryption functionality
RUN python3 -c "import pyarrow.parquet.encryption as ppe; print('Parquet encryption OK')"

# Default shell for the build stage
WORKDIR /arrowdev
CMD ["bash"]

# --- end of Dockerfile
```

Appendix 2: The connection_config file

The `connection_config` file is a file passed from the Python App down to the DBPS Agent being instantiated. The file is not read by the Arrow code. Currently, only the DBPS Remote Agent (i.e. libdbpsRemoteAgent.so) requires the `connection_config` file.

The file looks as follows.

```JSON
{
  "server_url": "http://192.168.1.86:18080",
  "connection_pool.max_pool_size": 23,
  "connection_pool.borrow_timeout_milliseconds": 100,
  "connection_pool.max_idle_time_milliseconds": 60000,
  "connection_pool.connect_timeout_seconds": 5,
  "connection_pool.read_timeout_seconds": 20,
  "connection_pool.write_timeout_seconds": 20,
  "connection_pool.num_worker_threads" : 24
}
```

Config parameter	Meaning
`server_url`	Base URL (scheme/host/port) of the DBPS remote agent to connect to. Used as the target for all HTTP requests.
`connection_pool.max_pool_size`	Maximum number of HTTP client connections the pool is allowed to keep/hand out concurrently. Caps parallel in-flight requests and resource usage.
`connection_pool.borrow_timeout_milliseconds`	How long a caller waits to obtain a connection from the pool when all are busy. If exceeded, the request fails with a timeout/acquire error.
`connection_pool.max_idle_time_milliseconds`	How long an unused (idle) connection may remain in the pool before being closed. Helps avoid keeping stale connections and reduces resource use.
`connection_pool.connect_timeout_seconds`	Timeout for establishing a TCP connection (and any handshake needed to connect). Prevents hanging during initial connect to the server.
`connection_pool.read_timeout_seconds`	Timeout for receiving data from the server once a request is sent. Triggers if the server is too slow to respond or stalls mid-response.

Config parameter	Meaning
`connection_pool.write_timeout_seconds`	Timeout for sending request data to the server. Triggers if the client can't write the request payload/headers promptly.
`connection_pool.num_worker_threads`	Number of worker threads used to run pooled request work concurrently. Typically aligns with expected parallelism (and often near CPU core count).